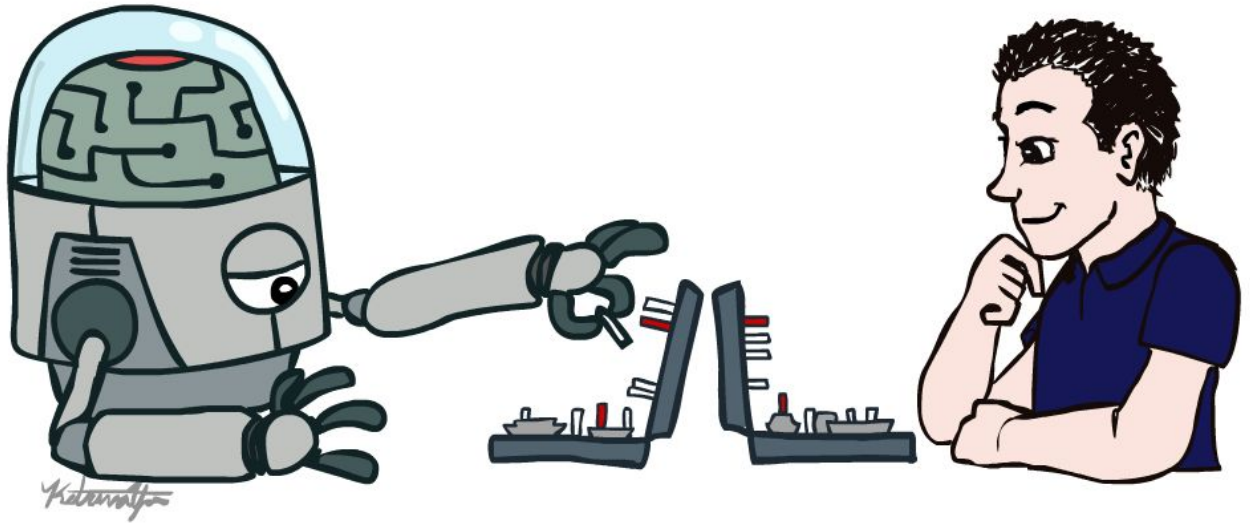
Computer Organization and Assembly Language
Week 2

Instructor: Aqsa Safdar

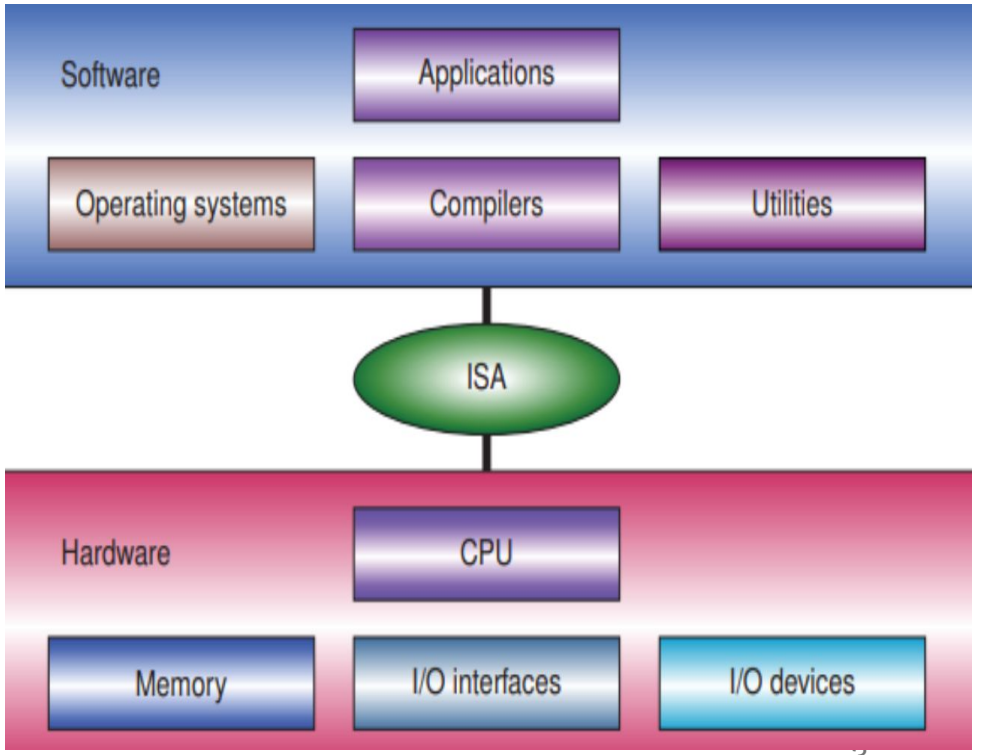


Overview

- Definition of ISA
- Role of ISA in computer organization
- Hardware–software relationship
- MIPS architecture overview
- RISC vs CISC
- MIPS instruction formats (R, I, J)

Instruction Set Architecture (ISA)

- The set of instructions and rules that define how software communicates with computer hardware.
- ISA is like the communication language between the brain and body, allowing software to control computer hardware.
- **ISA defines:**
  - Supported instructions (add, load, store, jump, etc.)
  - **Registers** available in the processor
  - Data types (byte, word, etc.)
  - Addressing modes
  - Memory access methods
- **Example**
  - High-level code: a = b + c
- **MIPS ISA instruction:**
  - add $t0, $t1, $t2
- ISA specifies how this operation is executed by hardware.



Contd….

- ISA provides a standard interface between programs and hardware.
- **Benefits:**
  - Software compatibility across processors
  - Hardware independence
  - Efficient instruction execution
- **Example:**
  - Programs written for **MIPS ISA** can run on different MIPS processors.

ISA is the interface between software and hardware because software uses ISA instructions and the hardware is designed to execute those instructions

Contd…

ISA acts as the bridge between architecture and organization. It determines:

- Instruction formats
- Register usage
- Memory addressing

So the ISA (Instruction Set Architecture) defines **what the computer can do from the programmer's point of view**.

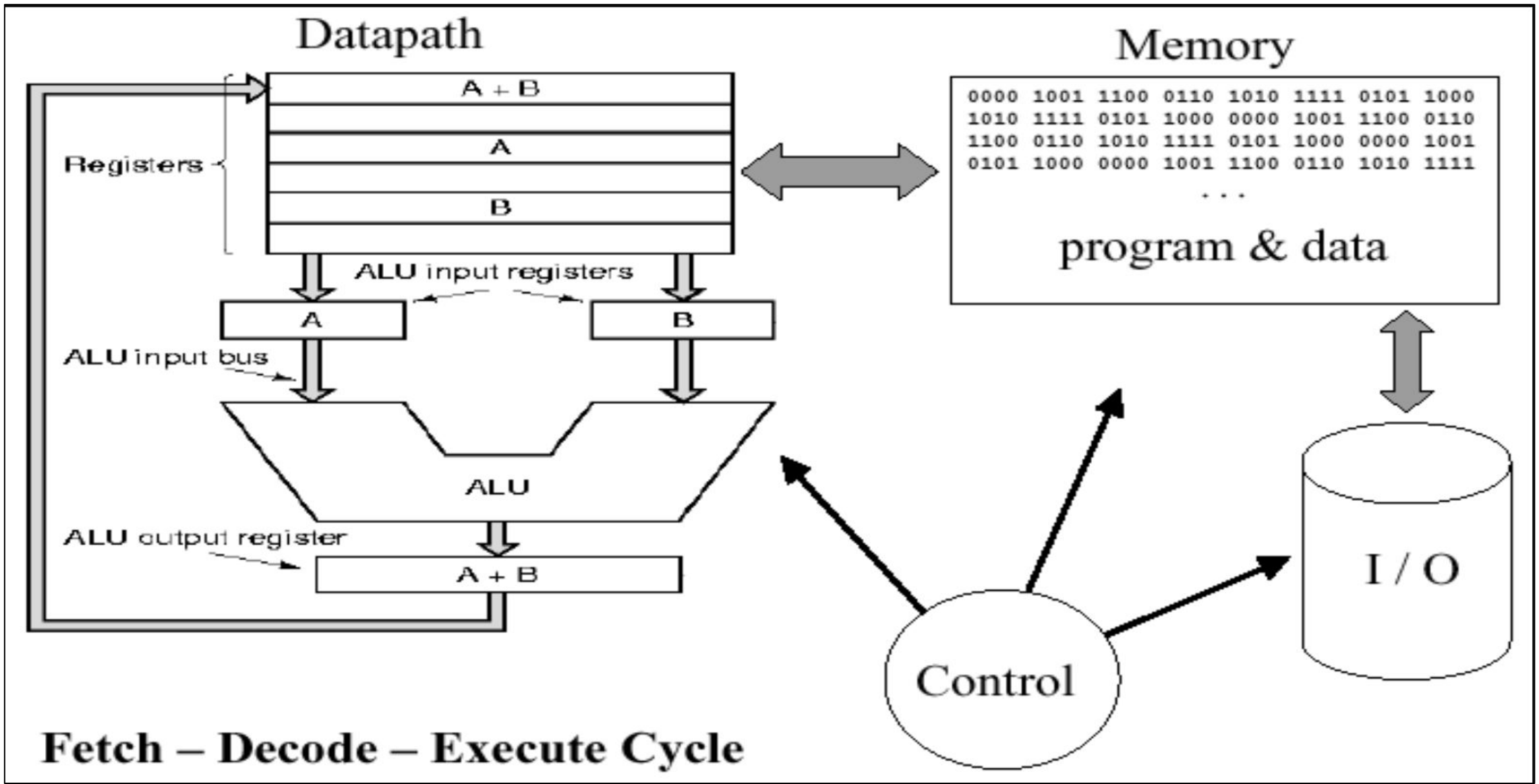
Computer organization then implements

these using:

- **Datapath** → the path through which data moves inside the CPU
- **Control Unit** → controls and coordinates operations
- **ALU (Arithmetic Logic Unit)** → performs arithmetic and logical operations

Why ISA is Important

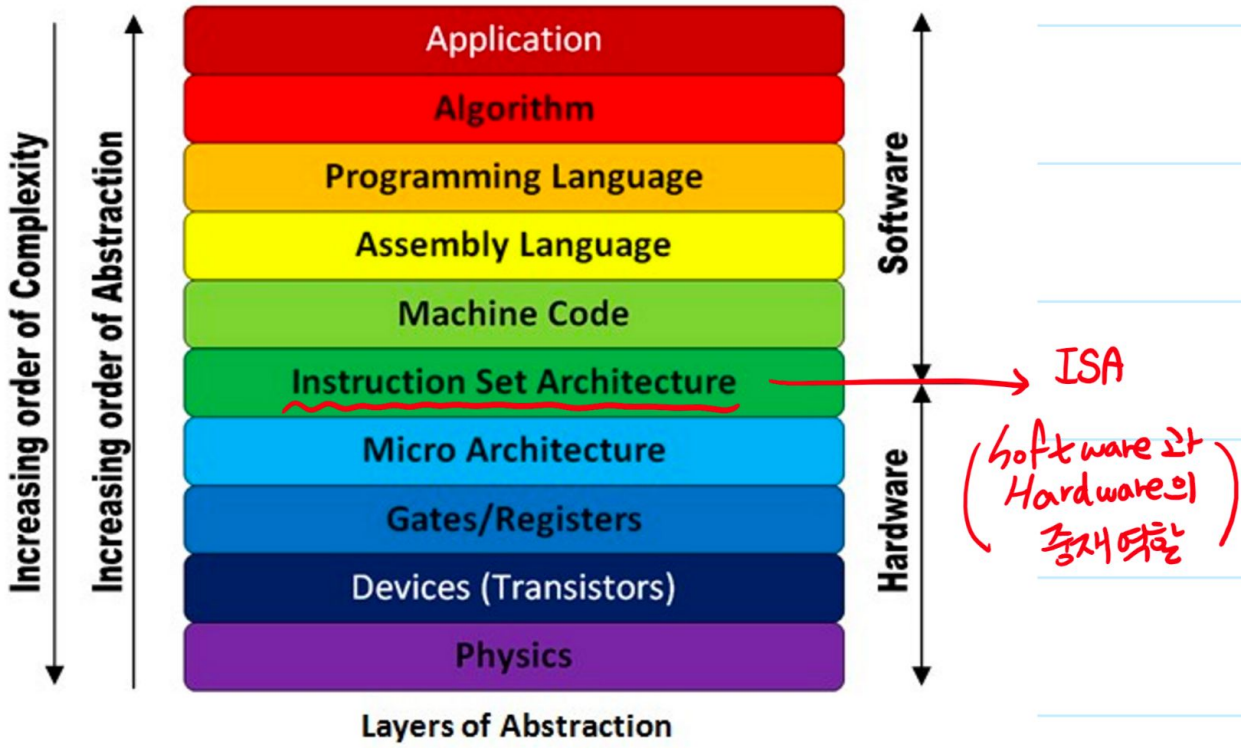
- ISA acts as a **bridge between architecture and organization**:
- **ISA defines what operations are possible**
- **Computer organization defines how those operations are carried out in hardware**


Datapath
A + B
A
B
Registers
ALU input registers
A
B
ALU input bus
ALU
ALU output register
A + B
Memory
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
. . .
program & data
Control
I / O
Fetch – Decode – Execute Cycle

- Programs written in high-level languages are translated into ISA instructions.
- **Example (**C code)
  - x = y + z
- **Assembly (MIPS)**
  - add $t0, $t1, $t2
  - $t1 contains y
  - $t2 contains z
  - $t0 Will store y + z

- Hardware executes this instruction

```
High-level code (C)
        ↓
Assembly / ISA instruction
        ↓
CPU hardware executes it
```

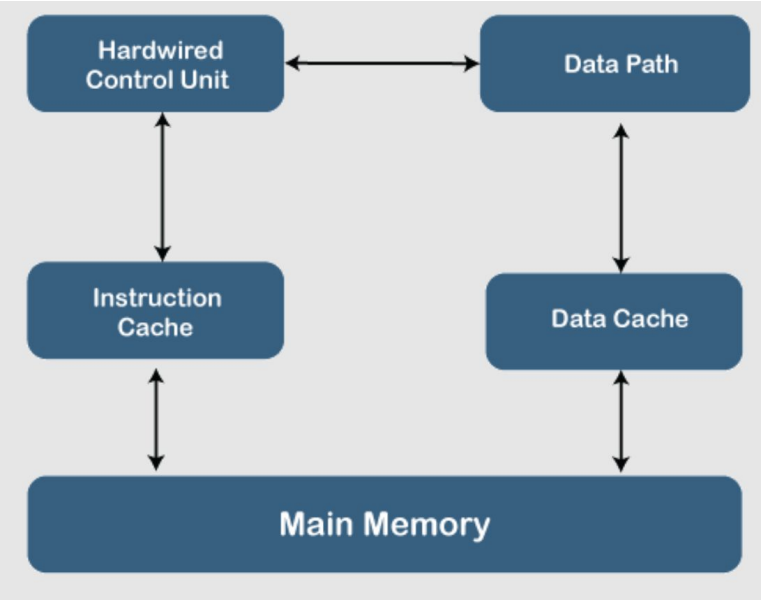


Computer System Equivalent

Human Body	Computer System
Brain	CPU
Nervous system	Instruction Set Architecture (ISA)
Muscles/organs	Hardware components (ALU, registers, memory)
Command from brain	Program instruction
Action	Instruction execution

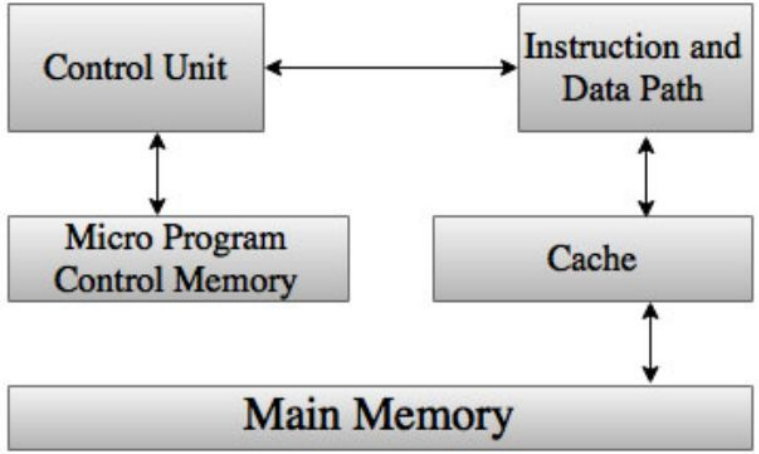
RISC (Reduced Instruction Set Computer)

- Simple instructions
- Fixed instruction length
- Fast execution
- Load/store architecture
- **Example processors:**
  - MIPS
  - ARM
  - RISC-V



CISC (Complex Instruction Set Architecture)

- Complex instructions
- Variable instruction length
- More addressing modes
- **Example processors:**
  - Intel x86
  - AMD processors



RISC vs CISC

Feature	**RISC**	**CISC**
Instruction complexity	Simple	Complex
Instruction length	Fixed	Variable
Execution speed	Faster	Slower
Example	MIPS	x86

RISC vs CISC

Feature	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Definition	Complex Instruction Set Computer	Reduced Instruction Set Computer
Number of Instructions	Large number of instructions	Fewer number of instructions
Instruction Format	Variable length instruction format	Fixed length instruction format
Addressing Modes	Large number of addressing modes	Few addressing modes
Cost	Higher cost	Lower cost
Processing Power	More powerful instructions	Less powerful individual instructions
Instruction Execution	Multiple clock cycles per instruction	Usually single clock cycle per instruction
Memory Access	Instructions can directly manipulate memory	Operations performed mainly in registers
Control Unit	Microprogrammed control unit	Hardwired control unit
Examples	Intel x86, Motorola 68000, Mainframe processors	MIPS, ARM, SPARC, Fujitsu

MIPS

- MIPS stands for Microprocessor without Interlocked Pipeline Stages
- **Key features:**
  - RISC architecture
  - Simple instruction format
  - Load/store architecture:
  - Fixed instruction length (32 bits)

Common Registers in MIPS

Register	**Purpose**
$t0–$t9	temporary
$s0–$s7	saved
$zero	special register in MIPS that always holds the value 0.
$ra	return address

Instruction Formats in MIPS

- **What is an Instruction Format?**
- An **instruction format** defines the **layout of bits within an instruction**.
- It specifies:
  - Operation to perform
  - Source operands
  - Destination operand
  - Immediate values or addresses

MIPS

- MIPS follows **Load/Store architecture**. Only **load and store instructions access memory**
- Arithmetic operations use registers only
- **Example:**
  - lw $t0, 0($t1)
  - add $t2, $t0, $t3
  - sw $t2, 0($t1)
- **Steps:**
  - Load value from memory
  - Perform computation
  - Store result back to memory

MIPS Instruction Formats

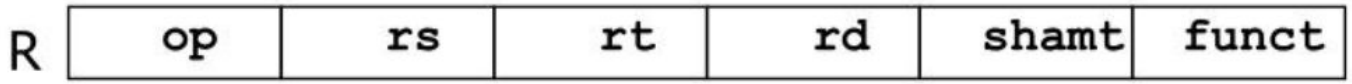
- Each MIPS instruction has **32 bits**.
- Three main formats:
  - R-type (Register)
  - I-type (Immediate)
  - J-type (Jump)
- MIPS Instruction (32 bits)

|---- R-Type
|---- I-Type
|---- J-Type

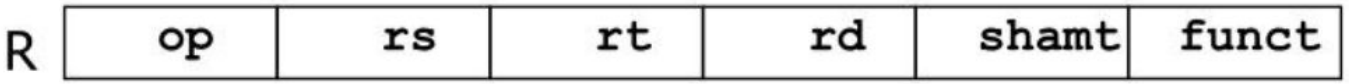
R- Type

- Used for **register-to-register operations**. That means:
  - Data is **taken from registers**
  - Operation is performed in **ALU**
  - Result is **stored back in a register**
- **Example:**
- add $t0, $t1, $t2
- **Meaning:**
- $t0 = $t1 + $t2
- **Format**

  | opcode | register source | register target | register destination | shamt | funct |
- **Suggested Diagram**
- |6|5|5|5|5|6|
  opcode rs rt rd shamt funct



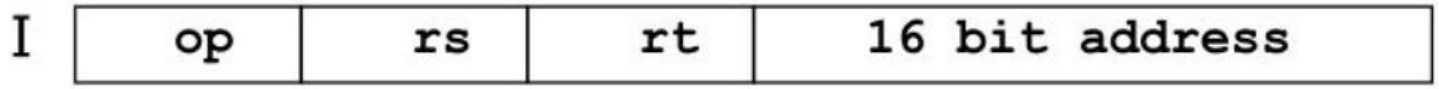
R- Type



Field	Bits	Purpose
opcode	6	Defines instruction type (for R-type usually 000000)
rs	5	First source register
rt	5	Second source register
rd	5	Destination register
shamt	5	Shift amount (used in shift instructions)
funct	6	Specifies the exact operation (add, sub, and, etc.)

I-Type

- **Used for:**
  - memory access with offset
  - immediate values
  - branch instructions

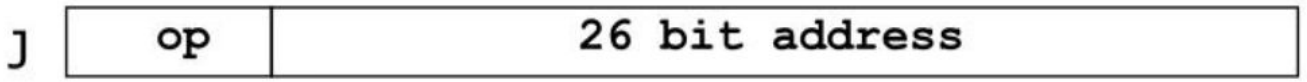


- **Example:**
  - lw $t0, 4($t1)
  - Meaning: Load value from memory.
- **Format**

```
| opcode |    rs   |    rt   |    immediate |
|  6bit  | 5bit    | 5bit    |    16bit     |
```

J- Type

- Used for **jump instructions**.
- **Example:**
  - j loop
  - j 0x00400010
- **Meaning:**
  - Jump to instruction labeled **loop**.
- **Format**
  - | opcode | address |
  - **opcode (6 bits)** → tells the CPU it's a jump instruction
  - **address (26 bits)** → target address to jump to



Field	Value
opcode	add
rs	$t1
rt	$t2
rd	$t0

- **Instruction:**
  - add $t0, $t1, $t2
- **This tells the CPU:**
  - $t0 = $t1 + $t2

Example

Opcode (Operation Code)

- **Size:** 6 bits
- **Purpose:** Tells the processor **what type of instruction it is**.
- In **R-format instructions**, the opcode is usually **000000**.
- **Example:**
  - add $t0, $t1, $t2
  - Opcode = 000000
  - This indicates that the **exact operation will be determined by the funct field**.

rs (Source Register)

- **Size:** 5 bits
- **Purpose:** Specifies the **first source register** containing data.
- **Example:**
  - add $t0, $t1, $t2
- **Here:**
  - rs = $t1
- **Meaning:** take the first operand from register **$t1**.

rt (Target Register)

- **Size:** 5 bits
- **Purpose:** Specifies the **second source register**.
- **Example:**
  - add $t0, $t1, $t2
- **Here:**
  - rt = $t2
- **Meaning:** take the second operand from register **$t2**.

rd (Destination Register)

- **Size:** 5 bits
- **Purpose:** Specifies the **register where the result will be stored**.
- **Example:**
  - add $t0, $t1, $t2
- **Here:**
  - rd = $t0
- **Meaning:** store the result in **$t0**.

shamt (Shift Amount)

- **Size:** 5 bits
- **Purpose:** Used **only for shift instructions** to indicate **how many positions to shift**.
- **Example:**
  - sll $t0, $t1, 2
- **Here:**
  - shamt = 2
- **Meaning:**
  - shift the value in **$t1** left by **2 bits** and result store in $t0.
  - For most other instructions (like add, sub), **shamt = 00000**.

Instruction	funct value
add	100000
sub	100010
and	100100
or	100101
slt	101010

funct (Function Code)

- **Size:** 6 bits
- **Purpose:** Specifies the **exact operation** when opcode = 000000.
- **Examples:**
  - So the processor checks the **funct field** to know which operation to perform.

Field	Meaning
opcode	000000
rs	$t1
rt	$t2
rd	$t0
shamt	00000

Example

- **Instruction:**
  - add $t0, $t1, $t2

Immediate Field in I-Type Format

- **I-Type Format**
  - | opcode | rs | rt | immediate |
    6 bits 5 5 16 bits
- **What is Immediate?**
  - The **immediate field** is a **16-bit constant value** that is included directly inside the instruction.
  - Instead of taking the value from another register, the processor uses this constant value during execution.
- **Purpose**
  - The immediate value is used for:
  - arithmetic operations
  - memory addressing
  - branch instructions

Example 1: Arithmetic Immediate

- addi $t0, $t1, 5
- **Meaning:**
  - $t0 = $t1 + 5
- **Here:**
  - rs = $t1
  - rt = $t0
  - immediate = 5
- So the processor adds **5 directly** to the value in $t1.

Example 2: Memory Access

- lw $t0, 8($t1)
- **Meaning:**
  - $t0 = Memory[$t1 + 8]
- **Here:**
  - $t1 holds the base address
  - 8 is the **immediate offset**
- The CPU calculates the final memory address by:
  - base register + immediate value

Address Field in J-Type Instruction

- **What is the Address Field?**
  - The **address field** is a **26-bit value** used to specify the **target location to jump to** in the program.
  - It tells the processor **where the next instruction should come from**.
- **Purpose**
  - It is used for **jump instructions**.
- Examples:
  - j
  - jal

Example

- j loop
- **Meaning:**
  - Jump to the instruction labeled **loop**.
- If the address stored in loop is:
  - 00400040
- The processor updates the **Program Counter (PC)** to that location.

How the Jump Address is Calculated

- The **26-bit address is not the full address**, so the processor combines it with the **upper bits of the Program Counter (PC)** to create the final 32-bit address.
- **Simplified idea:**
  - Final Address = PC upper bits + 26-bit address + 00
  - (The 00 is added because instructions are word aligned. (every instruction is 4 bytes, so the last 2 bits are always 0).)

Summary

- ISA defines communication between hardware and software
- MIPS is a RISC architecture
- Load/store model used in MIPS
- Instruction formats include R, I, and J types
- ISA acts as the bridge between programs and hardware